

Rewrite plan + learning roadmap

Strategy: Don't cold-rewrite 50k lines of C++. Bind the C API first — exactly like tritonserver-rs did for Rust — add safe Zig abstractions on top, then replace internals incrementally as you understand the system. You learn and ship simultaneously.

Phases (6 phases · ~30 weeks)

phase 0 Zig fundamentals

Wk 1-3

You know Go (GC, channels) and basic C. Zig sits between them — manual memory with an Allocator interface, no hidden allocations, compile-time power through comptime. Steep for a week, then it clicks.

- error unions (!T)
- Allocator interface
- comptime generics
- defer / errdefer
- build.zig + linking

Example code:

```
const std = @import("std");
// Error handling: Go (T, error) → Zig !T (error union)
pub fn parseShape(alloc: std.mem.Allocator, dims: []const i64) ![]usize {
    const shape = try alloc.alloc(usize, dims.len);
    errdefer alloc.free(shape); // only runs on error paths
    for (dims, 0..) |d, i| {
        if (d < 0) return error.NegativeDim;
        shape[i] = @intCast(d);
    }
    return shape; // caller owns this allocation
}

// comptime: generate specialized types at zero runtime cost
pub fn Tensor(comptime T: type) type {
    return struct {
        data: []T,
        shape: []const usize,
        pub fn byteLen(s: @This()) usize {
            return s.data.len * @sizeof(T);
        }
    };
}
```

■ Milestone: CLI tool that parses a Triton model config.pbtxt and prints tensor shapes with dtypes

phase 1 C bindings layer

Wk 3-6

Triton's entire core is a C API (tritonserver.h, ~2000 lines). Zig imports C headers directly with @cImport — no binding generators, no wrapper glue files. This is your foundation for everything else.

- @cImport / @cInclude
- callconv(.C) ABI

- ?*anyopaque
- extern struct layout
- [:0]u8 null-terminated strings

Example code:

```
const c = @cImport({ @cInclude("tritonserver.h"); });
pub const TritonErr = error{ InvalidArg, NotFound, Unavailable, Unknown };
// Wrap the C error convention: null pointer = success
pub fn check(err: ?*c.TRITONSERVER_Error) TritonErr!void {
    const e = err orelse return;
    defer c.TRITONSERVER_ErrorDelete(e);
    return switch (c.TRITONSERVER_ErrorCode(e)) {
        c.TRITONSERVER_ERROR_INVALID_ARG => error.InvalidArg,
        c.TRITONSERVER_ERROR_NOT_FOUND => error.NotFound,
        else => error.Unknown,
    };
}

// Map C enum to Zig enum – comptime-safe, zero overhead
pub const DataType = enum(c_uint) {
    fp32 = c.TRITONSERVER_TYPE_FP32,
    fp16 = c.TRITONSERVER_TYPE_FP16,
    int8 = c.TRITONSERVER_TYPE_INT8,
};
```

■ Milestone: Zig program that initializes a `TRITONSERVER_Server`, queries liveness, and shuts down cleanly

phase 2 Server & model repo

Wk 6–10

Build idiomatic Zig wrappers for the server lifecycle. The `init/deinit` ownership pattern replaces C++ RAII. Learn filesystem traversal and JSON parsing to load model configs from the repository.

- `init/deinit` ownership
- `std.fs` traversal
- `std.json` parsing
- `std.ArrayList`
- `dupeZ` for C strings

Example code:

```
pub const Server = struct {
    handle: *c.TRITONSERVER_Server,
    alloc: std.mem.Allocator,
    pub fn init(alloc: std.mem.Allocator, model_repo: []const u8) !Server {
        var opts: ?*c.TRITONSERVER_ServerOptions = null;
        try check(c.TRITONSERVER_ServerOptionsNew(&opts));
        defer c.TRITONSERVER_ServerOptionsDelete(opts);
        const repo_z = try alloc.dupeZ(u8, model_repo);
        defer alloc.free(repo_z);
        try check(c.TRITONSERVER_ServerOptionsSetModelRepositoryPath(opts, repo_z));
        var handle: ?*c.TRITONSERVER_Server = null;
        try check(c.TRITONSERVER_ServerNew(&handle, opts));
        return .{ .handle = handle.?, .alloc = alloc };
    }
    pub fn deinit(self: *Server) void {
        _ = c.TRITONSERVER_ServerDelete(self.handle);
    }
};
```

```
};
```

- Milestone: Load a real model repo, list available models with typed tensor metadata and batch limits

phase 3 Request pipeline

Wk 10–16

The core loop: accept a request, send it async to the backend, handle the C callback on completion. Lifetimes, thread safety, and opaque pointer patterns all converge here. This is where Zig shines.

- C fn pointer callbacks
- ?*anyopaque user context
- std.Thread.Semaphore
- @ptrCast/@alignCast
- callconv(.C)

Example code:

```
pub const InferCtx = struct {
    sem: std.Thread.Semaphore = .{},
    resp: ?InferenceResponse = null,
    // Called from Triton's thread pool – must match C ABI exactly
    pub fn onComplete(
        raw: ?*c.TRITONSERVER_InferenceResponse,
        _: u32,
        userp: ?*anyopaque,
    ) callconv(.C) void {
        const ctx: *InferCtx = @ptrCast(@alignCast(userp));
        ctx.resp = InferenceResponse.fromRaw(raw.?) catch null;
        ctx.sem.post();
    }
};

pub fn inferSync(server: *Server, req: *Request) !InferenceResponse {
    var ctx = InferCtx{};
    try check(c.TRITONSERVER_ServerInferAsync(
        server.handle, req.handle, &InferCtx.onComplete, &ctx,
    ));
    ctx.sem.wait();
    return ctx.resp orelse error.InferenceFailed;
}
```

- Milestone: End-to-end: send a real inference request to a loaded model, decode typed output tensors

phase 4 Scheduler & batching

Wk 16–22

Dynamic batching is the key performance feature. Collect requests in a time window, merge into one large GPU batch. Requires lock-free queues and precise atomic memory ordering — the hardest Zig you will write.

- lock-free MPSC queue
- std.atomic.Value
- memory ordering (.acq_rel)
- std.Thread.Pool
- timer window flush

Example code:

```
pub fn MpscQueue(comptime T: type) type {
    return struct {
        const Node = struct { val: T, next: std.atomic.Value(*Node) = .{null} };
    };
}
```

```

head: std.atomic.Value(?*Node) = .{null},
tail: ?*Node = null,
pub fn push(q: *@This(), node: *Node) void {
const prev = q.head.swap(node, .acq_rel);
const link = if (prev) |p| &p.next; else { q.tail = node; return; };
link.store(node, .release);
}
pub fn pop(q: *@This()) ?T {
const tail = q.tail orelse return null;
q.tail = tail.next.load(.acquire);
return tail.val;
}
};
}

// 5ms batch window → dispatch to model instance
pub fn batchWorker(ctx: *BatchCtx) void {
while (!ctx.done.load(.acquire)) {
std.time.sleep(5 * std.time.ns_per_ms);
ctx.flushBatch();
}
}
}

```

■ Milestone: Batchers achieving >80% GPU utilization vs baseline single-request throughput

phase 5 Backend plugin system

Wk 22–30

Triton loads backends (TensorRT, ONNX, etc.) as .so plugins at runtime. std.DynLib wraps dlopen/dlsym. Write a Zig-native backend, export the TRITONBACKEND C symbols, and watch it load inside a real server.

- std.DynLib (dlopen)
- @export for C symbols
- build.zig shared lib
- comptime vtable dispatch
- symbol visibility

Example code:

```

pub const Backend = struct {
lib: std.DynLib,
vtable: VTable,
const VTable = struct {
init: *const fn(*c.TRITONBACKEND_Backend) callconv(.C) ?*c.TRITONSERVER_Error,
};
pub fn load(path: [:0]const u8) !Backend {
var lib = try std.DynLib.open(path);
errdefer lib.close();
return .{ .lib = lib, .vtable = .{
.init = lib.lookup(@TypeOf(VTable.init),
"TRITONBACKEND_Initialize"
) orelse return error.MissingSymbol,
}};
}
};

// In your Zig backend .so – export this symbol with C ABI
export fn TRITONBACKEND_Initialize(
b: *c.TRITONBACKEND_Backend

```

```

) ?*c.TRITONSERVER_Error {
    _ = b;
    return null; // null = success in Triton's C convention
}

```

■ Milestone: A Zig-native pass-through backend .so that loads and runs inside a real Triton server

Low-level systems concepts to study

These are the foundational concepts you need to understand as you work through the phases. Ordered by how urgently you need them.

critical Memory model

Zig has no GC. Every allocation has an explicit owner and lifetime. The Allocator interface decouples what you allocate from how — crucial when tensor buffers can live on CPU heap, DMA-pinned memory, or GPU VRAM.

- Stack vs heap — when each makes sense (stack is free, heap has a cost)
- Arena allocator: allocate many things, free all at once — perfect for per-request lifetimes
- Fixed buffer allocator: zero-heap hot paths in the scheduler inner loop
- @sizeof / @alignOf — know exactly what your structs cost before allocating
- std.heap.c_allocator vs Zig allocators when data crosses the C ABI boundary

critical ABI and C interop

Triton's public interface is a ~2000-line C header. You need struct layout, calling conventions, and pointer semantics at the ABI level to wrap it without silent corruption.

- extern struct — C-compatible memory layout with no Zig field reordering
- callconv(C) — mandatory on every callback you pass into C code
- ?*anyopaque — type-erased user context pointer (C void*)
- dupeZ pattern — converting Zig slices to null-terminated C strings safely
- How to verify struct layout matches: @offsetof vs offsetof in C header

very important Concurrency

Triton runs N model instances in parallel OS threads. You need real threads, mutexes, semaphores, and atomics — not just async I/O. Harder than Go goroutines because you manage lifetimes.

- std.Thread.spawn, join, detach — OS threads, not green threads
- Mutex / RwLock — critical sections and read-heavy state (model metadata)
- Semaphore — signaling inference completion across threads (callbacks)
- std.atomic.Value — lock-free counters and shutdown flags
- Memory ordering: .acquire .release .acq_rel — what barriers they provide

important I/O model

Triton serves HTTP/REST and gRPC. Zig 0.16 ships std.io with io_uring on Linux. Understanding the event model matters for the network ingress layer.

- io_uring: submission/completion rings — why it halves syscall overhead vs epoll
- Zig 0.16 std.io: colorless async — same code, swappable I/O backends
- HTTP/2 and gRPC: multiplexed binary streams, flow control, trailers
- Fiber model vs thread-per-connection: tradeoffs for inference serving latency
- poll → epoll → io_uring: the evolution and when each matters

important Dynamic linking

Triton loads backends as .so plugins at runtime. std.DynLib wraps dlopen/dlsym. You also need to export C-compatible symbols from Zig backends — visibility is easy to get wrong.

- ELF format: sections, symbol table, relocations (why .so loading works)
- dlopen / dlsym / dlclose — what std.DynLib calls under the hood
- @export — make a Zig function visible to C code as a named symbol
- ABI stability: why C ABI is the standard for long-lived plugin interfaces
- RPATH and LD_LIBRARY_PATH — how the runtime linker finds your .so file

important **Tensor memory layout**

Triton inference is about moving typed multi-dimensional arrays. Row-major/column-major, strides, and GPU-accessible memory regions are all part of the memory API you will wrap.

- Row-major (C-order) vs column-major (Fortran-order) — which backends expect what
- Strides: indexing into non-contiguous tensor views without copying
- TRITONSERVER_MemoryType: CPU, CPU_PINNED (DMA-accessible), GPU
- CUDA unified memory and pinned allocations — managed via C API you already wrap
- Memory bandwidth vs compute: why batch size determines GPU utilization

Resources

LEARNING ZIG

Learning Zig (Karl Seguin) openmymind.net/learning_zig	Best for Go devs. Free, online, frequently updated — start here
zig.guide zig.guide	Comprehensive language reference, well organized by topic
Introduction to Zig (Pedro Park) pedropark99.github.io/zig-book	Deep treatment of memory model and type system
dudethebuilder (YouTube) youtube.com/@dude_the_builder	Short focused videos: comptime, allocators, interfaces, generics
scattered-thoughts.net scattered-thoughts.net	High-signal systems thinking and Zig internals
Andrew Kelley's blog andrewkelley.me	Zig creator — language philosophy and design decisions
Zig 0.16 async I/O deep-dive daily.dev - Zig async io_uring	Essential reading before touching the network layer

ZIG CODEBASES TO STUDY

TigerBeetle github.com/tigerbeetle/tigerbeetle	Gold standard production Zig — distributed financial database
Bun github.com/oven-sh/bun	JavaScript runtime — excellent C interop patterns at scale
zap (HTTP server) github.com/zigzap/zap	Fast HTTP in Zig — relevant to Triton REST layer
tritonserver-rs (Rust reference) github.com/3xMike/tritonserver-rs	Study the C wrapper layering approach before writing Zig

LOW-LEVEL SYSTEMS

CS:APP (Bryant & O'Hallaron)	The bible — memory hierarchy, linking, concurrency, I/O
---	---

csapp.cs.cmu.edu

The Linux Programming Interface

man7.org/tlpi

Is Parallel Programming Hard?

kernel.org — [perfbook](#)

Beej's Guide to Network Prog.

beej.us/guide/bgnet

Everything Linux at the syscall level — threads, I/O, signals

Free PDF — memory ordering, lock-free algorithms, RCU

Free — TCP/UDP, relevant for Triton HTTP/gRPC ingress

TRITON REFERENCE

Triton architecture docs (NVIDIA)

docs.nvidia.com — [Triton arch](#)

tritonserver.h (C API)

github.com/triton-inference-server/core —
[tritonserver.h](#)

tritonbackend.h (plugin API)

github.com/triton-inference-server/core —
[tritonbackend.h](#)

Read this before writing a single line of Zig

2000-line C API you will wrap

Backend plugin interface — Phase 5 target